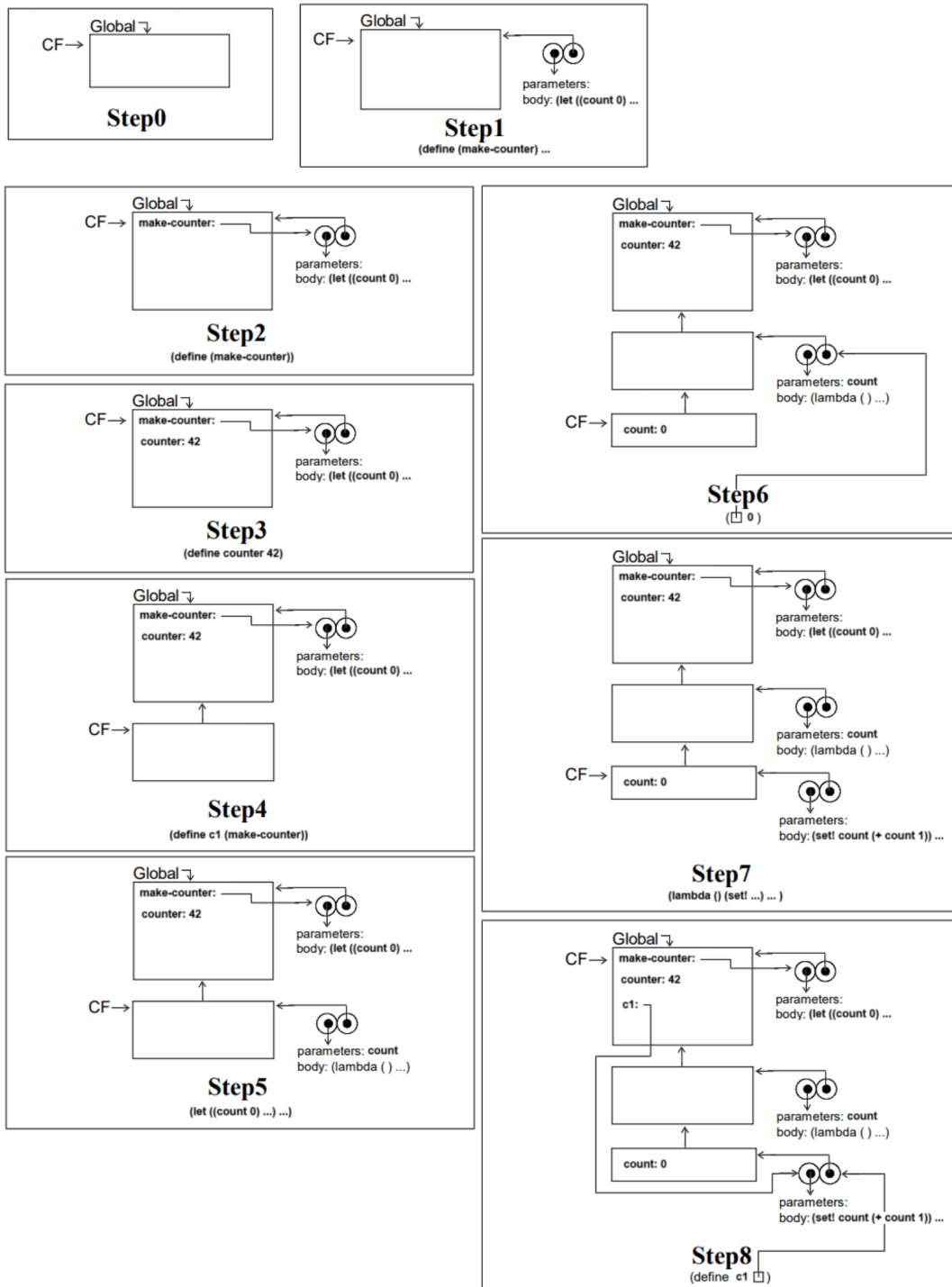
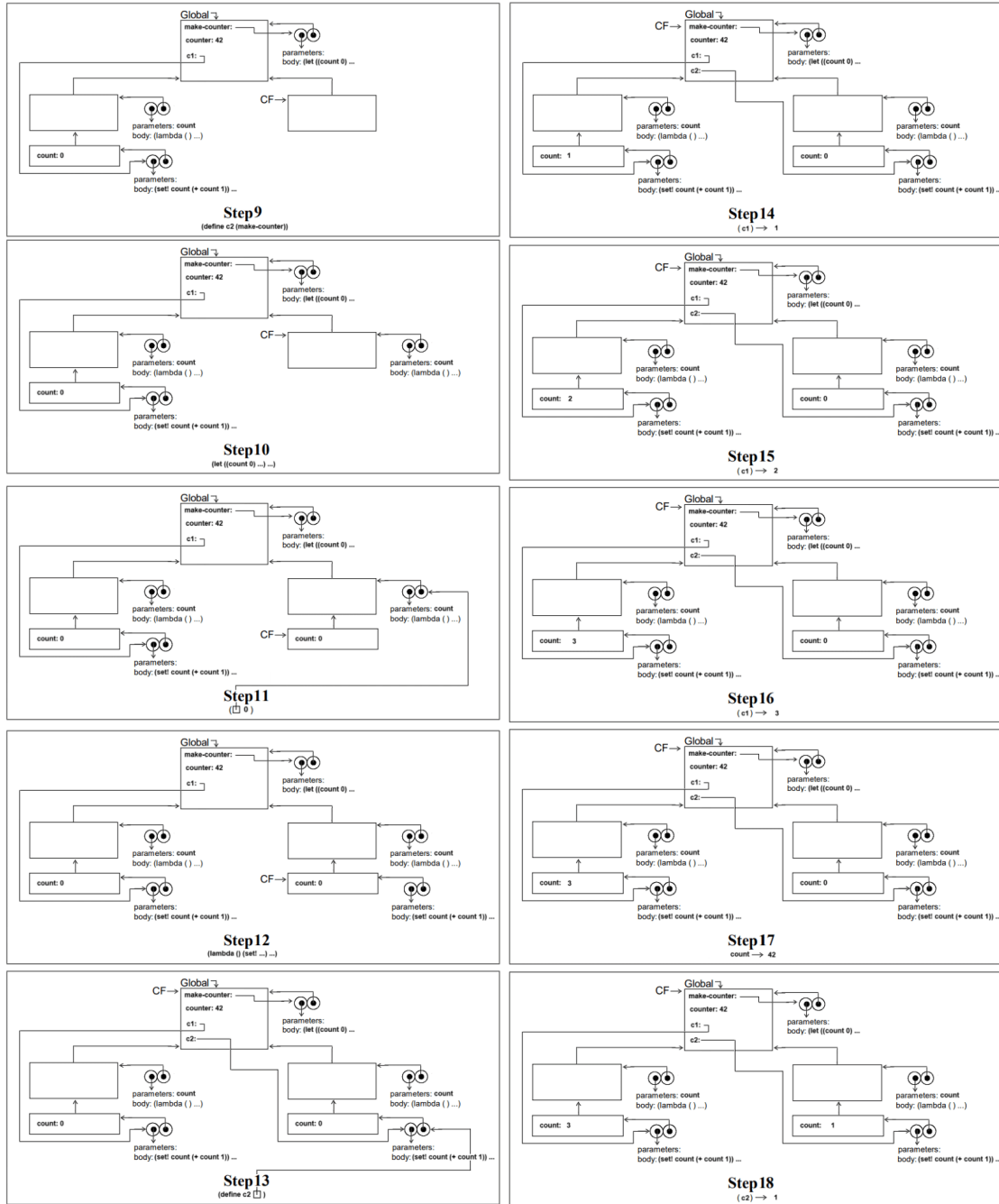


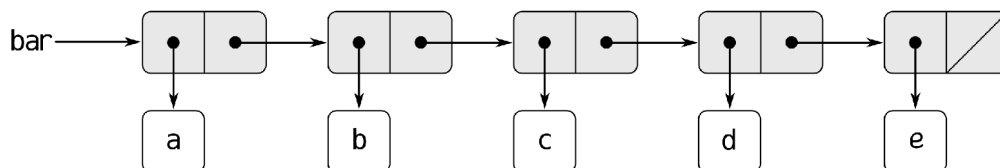
Oppgave 1b



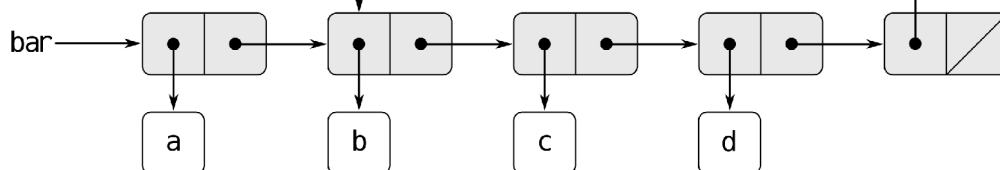


Oppgave 3a

bar etter define:



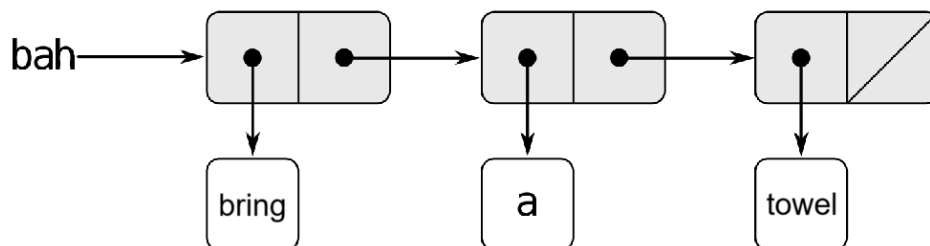
bar etter set-cdr!:



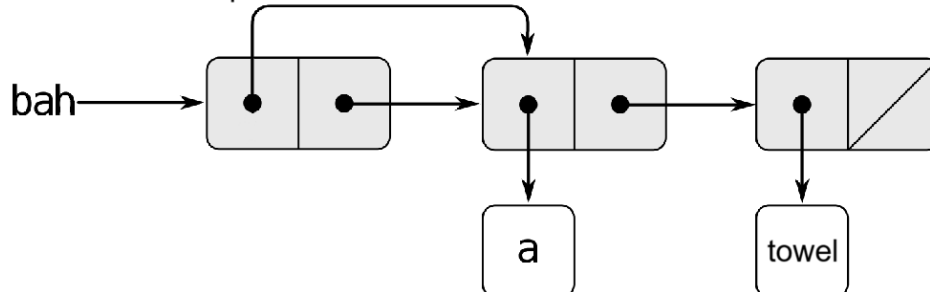
Vi får verdiene vi gjør ved kallene på list-ref på grunn av listen er nå en sirkulære liste. Dette gjør at listen er uendelig, og derfor nå kan vi kalle på høyere index enn det vi kunne før. Listen bar for de først 8 verdiene hadde sett ut (a b c d b c d b). Derfor når vi kaller på indeksene 0, 3, 4, og 5, får vi a, d, b, c.

Oppgave 3b

bah før første kallet på set-car!:



bah etter første kallet på set-car!:



bah evaluerer til verdien den gjør etter det siste kallet på set-car! på grunn av i det siste kallet på set-car! endrer vi den første verdien av den første veridien av bah. Med andre ord, vi bytter a med 42. Siden andre verdien av bah refererer til samme a-en, blir denne også endret til 42, og derfor får vi ((42 towel) 42 towel) når vi kaller på bah.

Oppgave 3d

Predikatet `<list?>` fungerer på følgende måte:

- (1) Om input objekt er en tomt liste `()` eller `'null'`, returnerer det `#t`.
- (2) Om input objekt er et par `'(a . b)'`, og `'b'` er en ordentlig liste, returnerer det `#t`.
- (3) Hvis `'(a . b)'` er et par men `'b'` er ikke en ordentlig liste (altså er verken `'()` eller et par), så returnerer det `#f`.

Når vi kaller `<list?>` for *bah*, itererer predikatet gjennom (2) frem til det er på paret (towel `'()`). Siden dette er en ordentlig liste, og `'b'` i dette tilfelle er tomt, så returnerer det `#t`.

Når vi kaller `<list?>` for *bar*, itererer predikatet gjennom (2) frem til det er på paret (d ...) . Siden `'b'` i dette tilfelle refererer tilbake til en tidligere del av strukturen, oppdager predikatet en uendelig rekursjon og returnerer `#f`.